

Part 3 Video Script (Detailed Walkthrough)

Use this as a speaking guide for your recording.

1. Introduction (30-60 seconds)

Hello, my name is [Your Name].

This is Part 3 of my Global Logistics Management System project.

In this demo, I will show:

1. How I refactored the system into a Service-Oriented Architecture.
2. My backend Web API with Swagger and JWT authentication.
3. The MVC frontend consuming the API via HttpClient.
4. Automated API integration testing.
5. Full Docker Compose containerization with SQL Server, API, and frontend.

2. Solution Structure (60-90 seconds)

I will start by showing the solution structure.

The solution now has:

- GLMS.Web: MVC frontend project
- GLMS.Api: backend Web API project
- GLMS.Tests: unit and integration tests

I explain that in Part 2 the app was monolithic, but now the backend and frontend are decoupled. The API is the only layer that communicates with the database.

3. Backend API + JWT + Swagger (2-3 minutes)

Now I run GLMS.Api and open Swagger.

I show the available endpoints such as:

- GET /api/contracts with filtering
- POST /api/contracts
- PATCH /api/contracts/{id}/status
- Client and service request endpoints

Then I demonstrate authentication:

1. Use POST /api/auth/token with configured credentials.
2. Copy the JWT token from the response.
3. Click Authorize in Swagger and add Bearer token.
4. Execute protected endpoints successfully.

I explain that JWT secures API access and is required by the Part 3 brief.

4. Frontend MVC Consuming API (2-3 minutes)

Next I run GLMS.Web and show the UI.

I explain that controllers no longer use direct DbContext database access for business data operations. Instead, they call backend API endpoints through HttpClient.

I demonstrate:

- Viewing contracts list

- Creating a contract
- Updating workflow-relevant status via API flow
- Creating a service request linked to a contract

I point out the separation of layers:

- Presentation layer: MVC
- Service/data layer: API

5. Automated Integration Testing (2 minutes)

I open Test Explorer and run tests.

I show both existing unit tests and new integration tests.

I highlight the integration test scenario:

- Authenticate and receive JWT
- Call GET /api/contracts
- Assert HTTP 200
- Assert response JSON is not null

I explain why this matters in CI/CD:

- Tests detect regressions early
- Failing builds stop broken deployments
- Higher release confidence

6. Docker Compose Demonstration (2-3 minutes)

Now I show containerization.

I run docker compose up --build from project root.

I explain the three containers:

1. sql-server-db
2. glms-backend-api
3. glms-frontend-web

Then I show:

- Docker Desktop with all containers running
- Frontend in browser on mapped port
- API Swagger in browser on mapped port

I mention internal container networking allows frontend to communicate with API and API to communicate with SQL Server.

7. Technical Reflection (60-90 seconds)

I conclude with key reflection points:

- Automated testing is essential in CI/CD because it prevents breaking changes from reaching production.
- Docker ensures environment consistency across development, testing, and production.
- This solves the "it works on my machine" issue by running the same images everywhere.
- The system is now ready for future scaling and cloud-native deployment.

8. Closing (20-30 seconds)

That concludes my Part 3 demonstration.

I have shown the SOA refactor, secured API, frontend integration, automated testing, and full containerization.

Thank you.